

Robust neural circuit reconstruction from serial electron microscopy with convolutional recurrent networks

Drew Linsley^{1,†}, Junkyung Kim^{1,†}, David Berson², and Thomas Serre¹

¹Department of Cognitive, Linguistic & Psychological Sciences

²Carney Institute for Brain Science

Brown University, Providence, RI

{drew.linsley, junkyung_kim, david.berson, thomas-serre}@brown.edu

Abstract

Recent successes in deep learning have started to impact neuroscience. Of particular significance are claims that current segmentation algorithms achieve “super-human” accuracy in an area known as connectomics. However, as we will show, these algorithms do not effectively generalize beyond the particular source and brain tissues used for training – severely limiting their usability by the broader neuroscience community. To fill this gap, we describe a novel connectomics challenge for source- and tissue-agnostic reconstruction of neurons (STAR), which favors broad generalization over fitting specific datasets. We first demonstrate that current state-of-the-art approaches to neuron segmentation perform poorly on the challenge. We further describe a novel convolutional recurrent neural network module that combines short-range horizontal connections within a processing stage and long-range top-down connections between stages. The resulting architecture establishes the state of the art on the STAR challenge and represents a significant step towards widely usable and fully-automated connectomics analysis.

1. Introduction

Connectomics – the comprehensive mapping of synaptic circuits – is poised to revolutionize our understanding of neural computations. Technical advances permit generation of high-resolution serial electron microscope (SEM) images spanning volumes hundreds of microns across. These can reveal synaptic associations among classes of neurons and advance our understanding of the circuits underlying specific neural computations. Analysis software and crowd-sourcing platforms are being developed to mine these in-

valuable and extremely large data sets, and initial results are impressive (see [9, 47, 50, 43] for reviews.)

For example, SEM imaging of the retina was used to explain the direction-selectivity of certain ganglion cells based on their asymmetric contacts with a particular type of inhibitory interneuron [48]. Various labs are spearheading applications in the thalamus and visual cortex of the mouse, as well as the fruit fly nervous system, among other model organisms [51, 22, 2, 25, 7, 40]. Yet despite its great promise, SEM has not gained widespread adoption among neuroscientists.

One of the key steps in connectomics involves segmenting neurons from their surroundings – a task that requires such time intensive manual annotation that it is in effect impossible for all but the most basic organisms [53]. Computer vision has been critical to improving the efficiency of neuron segmentation and enabling the reconstruction of complex nervous systems at “superhuman” levels of accuracy [25]. These successes hint at the possibility of automating neuron reconstruction in SEM volumes at scales that would be unimaginable for manual work. As an example, it has been reported that computer vision permits a 250-fold reduction in processing time over brute-force manual annotation for a $100^3\text{-}\mu\text{m}^3$ tissue volume [18]. However, the 400 man-hours of expert annotation required for the generation of this system’s training data still represents a significant barrier. For computer vision to be most useful in practice, segmentation algorithms must generalize across brain tissues to allow completely automated mapping of neural circuits.

Consider Fig. 1a, which depicts sample SEM images from a variety of connectomics challenges [2, 20]. Current state-of-the-art approaches [25, 18] are trained and tested on disjoint sections of the same volumes (Fig. 1b). However, we have found that when asked to make judgments on different tissues (Fig. 1c), these network architectures do not

[†]These authors contributed equally to this work.

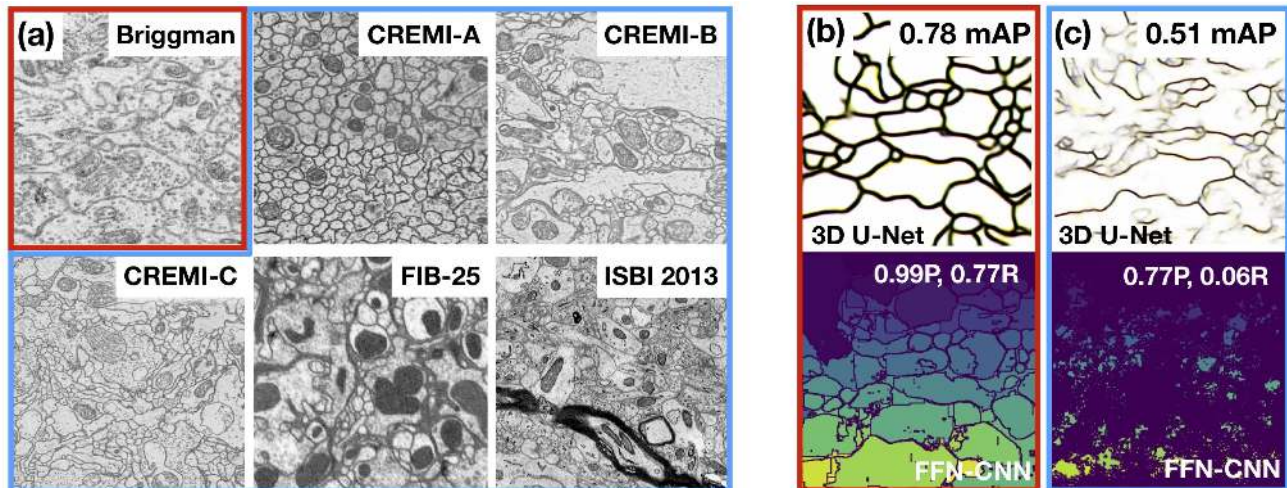


Figure 1. State-of-the-art deep learning architectures [25, 18] for neuron reconstruction generalize to held-out images from the same SEM volume but not to different volumes. (a) Representative SEM images. Samples collected from the fruit fly nervous system (Volumes A, B, and C volumes from CREMI, and FIB-25), mouse cortex (SNEMI3D), and retina (Briggman [2]). Colored boxes indicate the dataset used to train the models that produced the results in (b) and (c). (b) The top-panel shows performance of a state-of-the-art 3D U-Net [25] trained and tested on distinct sections of the Briggman volume (red outline in a) to predict the locations of membranes. A leading approach to neuron reconstruction utilizes these predictions as scaffold for segmenting neurons with post-processing methods like watershed. The 3D U-Net makes high-quality predictions of the locations of membranes, as measured by the contour detection metric mean average precision (mAP [54]; calculated per-slice and then averaged; higher is better). An alternate approach to neuron reconstruction is the Flood-filling network (FFN) [18], which successfully segments the Briggman volume when trained and tested on distinct sections of it (lower panel). Its performance is measured by precision and recall (P, R; [15]; higher is better). (c) The STAR challenge involves training models on images from multiple datasets (blue outline in a), and test on images from an independent dataset (Briggman). This challenge severely strains membrane predictions of the 3D U-Net (top-panel of c) and segmentations from the FFN (bottom-panel of c), demonstrating their limited generalization.

generalize well. Hence, model weights successfully trained on one dataset must be “fine-tuned” on tissue-specific annotations from another dataset to make accurate predictions on those images. Most neurobiology laboratories do not have the resources to annotate and curate SEM datasets at this scale, and they do not have the computer vision expertise to retrain state-of-the-art computer vision systems for their specific brain volumes. The development of computer vision algorithms that can generalize across brain volumes represents the next frontier in automating neuron reconstructions.

Contributions Here, we address the poor generalization of computer vision systems in connectomics with a novel challenge: the source- and tissue-agnostic reconstruction (STAR; [STAR-challenge.github.io](https://github.com/STAR-challenge)). This challenge presents a “training” dataset consisting of five publicly available and annotated volumes of SEM images representing a variety of organisms and imaging configurations (CREMI, FIB-25, and SNEMI3D; see Table 1); evaluation is performed on an independent volume [2] annotated by our group. We will demonstrate that state-of-the-art systems for neuron reconstruction fail on the STAR challenge.

An alternative to the feedforward convolutional neural

networks which form the backbone of state-of-the-art segmentation architectures can be inferred from the organization of the visual cortex, where feedback connections between neurons mediate their highly complex interactions and are thought to form the basis for robust vision. Recent work motivated by biological considerations introduced the horizontal gated recurrent unit (hGRU), and found that such horizontal interactions between computational units support parameter-efficient solutions to contour detection tasks [28].

We build on this framework and introduce the feedback gated recurrent unit (fGRU), which generalizes the recurrent connections introduced in the hGRU to include not only horizontal connections within a layer but also top-down connections between processing layers. We find that fGRU models perform on par or better than state-of-the-art systems for neuron reconstruction when trained and tested on tissue images sampled from the same volumes. Importantly, fGRU models also significantly outperform these systems on the STAR challenge, where generalizing to an independent volume is needed.

2. Related work

Neuron reconstruction datasets The development of computer vision systems for connectomics has been largely supported by the public availability of SEM datasets paired with expert annotations. These include volumes sampled from fruit fly nervous system (CREMI, cremi.org; FIB-25, <http://janelia-flyem.github.io/>) and mouse visual cortex (SNEMI3D, <http://brainiac2.mit.edu/SNEMI3D/>). The proposed STAR challenge combines these datasets with one from mouse retina [2]. Each of the volumes was fixed and imaged using slightly different methods, but at least to the human eye, appear visually similar (Fig. 1; see Table 1 for dataset details).

Automated 3D neuron reconstruction One classic computer vision pipeline to neuron reconstruction involves the following three stages: First, cell membranes are detected; second, a watershed algorithm grows supervoxels within each cell membrane; third, an additional post-processing step agglomerates supervoxels into more accurate segmentation masks [33]. One of the leading architectures featuring such an approach uses a 3D U-Net architecture for membrane detection – resulting in “superhuman” segmentation accuracy in the SNEMI3D challenge [25]. This network architecture combines the encoder-decoder structure of a typical U-Net [38] with extensive “residual connections” [17] that mix information between different processing stages, and losses for predicting voxel affinities.

A more recent approach integrates membrane detection and segmentation into the same end-to-end trainable neural network architecture [18]. Flood-filling networks (FFNs) segment volumes one neuron at a time. To date, they have outperformed all other approaches in processing the FIB-25 dataset. The core of this model is a 3D CNN with residual connections, which receives two types of inputs: a 3D volume of SEM images, and a corresponding mask volume which is iteratively updated by the network to assign voxels from the volume to a neuron under consideration. In the first iteration, the network processes a small 3D crop (which is called the network’s “field of view”) of the volume centered around a randomly initialized X/Y/Z “seed” location. Updates to the mask volume inform the model where to move its field of view on the next processing step. This process continues until an entire cell has been labeled.

Recurrent vision models Recurrent neural networks (RNNs) have classically been used for modeling temporal sequences, but work over recent years has started to demonstrate their utility for general vision tasks. Successful RNN applications in vision include object recognition and super-resolution tasks, where convolution kernels are applied re-

cursively to increase processing depth without adding parameters [26, 27, 21]. A growing body of work has shown the benefits of introducing learning rules or connectivity patterns that are inspired by the anatomy and/or the physiology of the visual cortex into RNNs [45, 29, 55, 32, 28].

In particular, the horizontal gated recurrent unit (hGRU) aims to approximate a recurrent-network neuroscience model of contextual illusions [31], with a convolutional-RNN module that can be trained using routines from deep learning. The resulting module uses trainable gates, borrowed from the gated recurrent unit (GRU [4]), to control the integration of information over time. A single hGRU layer learned to solve contour detection tasks more efficiently than CNNs (with many times the number of parameters) by learning patterns of horizontal connectivity between units at different spatial locations and across feature channels.

The proposed feedback gated recurrent unit (fGRU) builds on the hGRU, extending it into a general-purpose module for learning feedback interactions – both short- (i.e., horizontal connections within a layer) and long-range (i.e., top-down connections between layers).

3. Feedback gated recurrent unit

Recurrent-network neuroscience model We briefly review the hGRU [28], which this work builds upon. The hGRU implements the dynamics of a recurrent-network model of horizontal interactions [31], which explained human psychophysical responses to contextual illusions. We begin by describing this model. Its units are indexed by their 2D position (x, y) and feature channel k . Neural activity is governed by the following RNN, which approximates Euler integration on the dynamical system described in [31]:

$$\begin{aligned} \mathbf{C}^I &= (\mathbf{W}^I * \mathbf{H}^{(2)}[t - 1]) \\ \mathbf{C}^E &= (\mathbf{W}^E * \mathbf{H}^{(1)}[t]) \\ H_{xyk}^{(1)}[t] &= \epsilon^{-2} \left[\xi X_{xyk} - (\alpha H_{xyk}^{(1)}[t - 1] + \mu) C_{xyk}^I[t] \right]_+ \\ H_{xyk}^{(2)}[t] &= \epsilon^{-2} \left[\gamma C_{xyk}^E[t] \right]_+. \end{aligned} \quad (1)$$

These equations describe the evolution of recurrent synaptic inputs and outputs $\mathbf{H}^{(1)}, \mathbf{H}^{(2)} \in \mathbb{R}^{W \times H \times K}$, which are influenced by the feedforward drive $\mathbf{X} \in \mathbb{R}^{W \times H \times K}$ (i.e., the neural response to a stimulus) over discrete timesteps, denoted by $\cdot[t]$. By separating contributions from synaptic inputs and outputs, the model is able to simulate different known forms of linear and non-linear inhibitory and excitatory interactions between neurons (see [31] for details), implemented by the kernels $\mathbf{W}^I$ and $\mathbf{W}^E$ on the input and output, respectively.

The activity $\mathbf{C}^I$ describes horizontal inhibitory interactions on neuron outputs, $\mathbf{H}^{(2)}$. Inhibition is a function of

Name	Tissue	Imaging	Resolution	Voxels (X/Y/Z/Volumes)
CREMI	Fruit fly	ssTEM [46]	$4 \times 4 \times 40\text{nm}$	$1250 \times 1250 \times 125 \times 3$
FIB-25	Fruit fly	FIBSEM [24]	$8 \times 8 \times 8\text{nm}$	$520 \times 520 \times 520 \times 1$
SNEMI3D [20]	Mouse cortex	mbSEM [8]	$6 \times 6 \times 29\text{nm}$	$1024 \times 1024 \times 100 \times 1$
Briggman [2] (Ours)	Mouse retina	SBEM [6]	$16.5 \times 16.5 \times 23\text{nm}$	$384 \times 384 \times 384 \times 1$

Table 1. SEM image volumes used in the STAR challenge. The training dataset consists of CREMI, FIB-25, and SNEMI3D, whereas the test dataset, Briggman, is a volume from [2] annotated by us.

both the output, via the $\mathbf{C}^I$ activity, and the input, via the $\mathbf{C}^E$ activity. This supports both linear and non-linear forms of inhibition; the latter of which is referred to as “shunting” (or divisive) inhibition [14]. Shunting inhibition means that the effect of the inhibition on the neuron will scale with its output activity (the more active the neuron the stronger the inhibition will be). On the other hand, excitation in the model is a linear function of $\mathbf{C}^E$, which describes horizontal excitatory interactions between units in $\mathbf{H}^{(1)}$. Linear and non-linear forms of inhibition are controlled by the parameters μ and α , respectively, whereas linear excitation is controlled by γ . The ReLU function is denoted by $[\cdot]_+ = \max(\cdot, 0)$, and ϵ is a time constant.

The hGRU module The hGRU (Fig. 2) extends the convolutional recurrent model described in Eq. 1 with learnable gates, which facilitates learning with gradient descent [28]. We describe a formulation of this model that includes recurrent batch-norm [5], which leads to more consistent training. This hGRU is governed by the following equations, which relax many of the constraints of Eq. 1 that were shown to be unnecessary for contour detection tasks (see [28] for details):

$$\mathbf{G}^{(1)}[t] = \sigma(\text{BN}(\mathbf{U}^{(1)} * \mathbf{H}^{(2)}[t-1]))$$

$$\mathbf{C}^{(1)}[t] = \text{BN}(\mathbf{W} * (\mathbf{G}^{(1)}[t] \odot \mathbf{H}^{(2)}[t-1]))$$

$$\mathbf{G}^{(2)}[t] = \sigma(\text{BN}(\mathbf{U}^{(2)} * \mathbf{H}^{(1)}[t]))$$

$$\mathbf{C}^{(2)}[t] = \text{BN}(\mathbf{W} * \mathbf{H}^{(1)}[t])$$

$$H_{xyk}^{(1)}[t] = \zeta(X_{xyk} - C_{xyk}^{(1)}[t](\alpha_k H_{xyk}^{(2)}[t-1] + \mu_k))$$

$$\tilde{H}_{xyk}^{(2)}[t] = \zeta(\tau_k H_{xyk}^{(1)}[t] + \beta_k C_{xyk}^{(2)}[t] + \gamma_k H_{xyk}^{(1)}[t] C_{xyk}^{(2)}[t])$$

$$H_{xyk}^{(2)}[t] = H_{xyk}^{(2)}[t-1](1 - G_{xyk}^{(2)}[t]) + \tilde{H}_{xyk}^{(2)}[t] G_{xyk}^{(2)}[t]$$

$$\text{where } \text{BN}(\mathbf{r}; \delta, \nu) = \nu + \delta \odot \frac{\mathbf{r} - \hat{\mathbb{E}}[\mathbf{r}]}{\sqrt{\widehat{\text{Var}}[\mathbf{r}] + \eta}}. \quad (2)$$

Here, the feedforward drive $\mathbf{X}$ may correspond to the activity from a preceding convolutional layer. As in the model described in Eq. 1, this hGRU contains input and output states ($\mathbf{H}^{(1)}$ and $\mathbf{H}^{(2)}$), that allow it to capture complex nonlinear interactions between units in $\mathbf{X}$. Updates to these

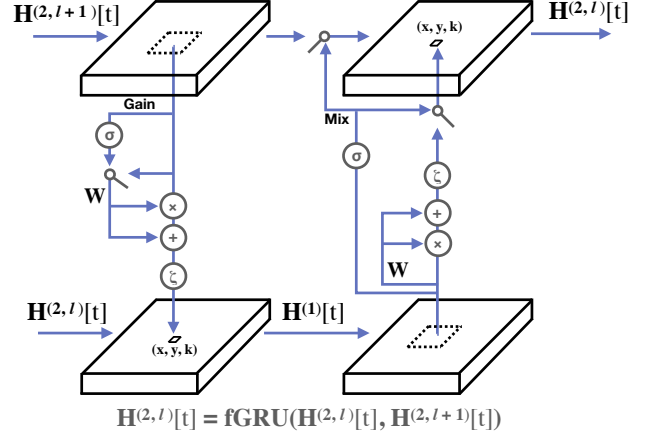


Figure 2. A diagram of the top-down module of the fGRU, which is a generalization of the hGRU [28] that allows for horizontal or top-down connectivity. The fGRU shares the same internal dynamics as the hGRU (captured by the function fGRU in the equation below the figure) but the input and output activities (the feed-forward drive $\mathbf{X}$ and the persistent output activity $\mathbf{H}^{(2)}[t-1]$ in the hGRU are replaced by $\mathbf{H}^{(2,l)}[t]$ (the persistent output at layer l) and $\mathbf{H}^{(2,l+1)}[t]$ (the persistent output at layer $l+1$), respectively. In the first stage, a gain is applied to $\mathbf{H}^{(2,l+1)}[t]$, and the resulting activity is convolved with the kernel $\mathbf{W}$ to compute top-down interactions between units in layer $l+1$ and l : both linear (+ symbol) and quadratic ($\times$ symbol). In the second stage, another set of linear and quadratic operations modulate an activity derived from convolving the updated $\mathbf{H}^{(1)}[t]$ with $\mathbf{W}$. A mix calculates the fGRU output by integrating this candidate output with $\mathbf{H}^{(2,l+1)}[t]$. Small solid-line squares within the circuit’s hypothetical activities (big boxes) denote the unit indexed by 2D position (x, y) and feature channel k . Dotted-line squares depict this unit’s projective field (a union of both classical and extra-classical definitions). Sigmoid and hyperbolic tangent point nonlinearities are denoted by σ and ζ , respectively.

hidden states are controlled by two activities: the “gain” $\mathbf{G}^{(1)}[t]$, which modulates channels in $\mathbf{H}^{(2)}[t-1]$ on every processing timestep, and the “mix” $\mathbf{G}^{(2)}[t]$, which mixes a candidate $\tilde{\mathbf{H}}^{(2)}[t]$ with the persistent $\mathbf{H}^{(2)}[t]$. Both the gain and mix are transformed into the range $[0, 1]$ by a sigmoid nonlinearity (σ). Horizontal interactions between input units are computed in $\mathbf{C}^{(1)}[t]$ by convolving $\mathbf{H}^{(2)}[t]$ with

the horizontal connection kernel $\mathbf{W}$. Note that unlike in the original model from Eq. 1, the hGRU no longer implements separate kernels for excitatory and inhibitory connections. Like Eq. 1, linear and quadratic interactions are controlled by the k -dimensional parameters μ and α (the latter of these corresponds to shunting inhibition described in Eq. 1). The pointwise function ζ is the squashing hyperbolic tangent that transforms activity into the range $[-1, 1]$. Separate applications of batch-norm are used on every timestep, where $\mathbf{r} \in \mathbb{R}^d$ is the vector of batch-normalized preactivations. The parameters $\delta, \nu \in \mathbb{R}^d$ control the scale and bias of normalized activities, η is a regularization hyperparameter, and $\odot$ is the elementwise multiplication.

Next, $\mathbf{H}^{(1)}[t]$ is convolved with the kernel $\mathbf{W}$ to compute $\mathbf{C}^{(2)}[t]$, which captures interactions between neighboring units in the newly-computed $\mathbf{H}^{(1)}[t]$. A candidate output $\tilde{\mathbf{H}}^{(2)}[t]$ is calculated via the k -dimensional parameters τ and β , which control the linear terms of this interaction, and γ which controls the quadratic contributions. Finally, the mix activity $\mathbf{G}^{(2)}$ is calculated by convolving $\mathbf{U}^{(2)}$ with $\mathbf{H}^{(1)}[t]$. After the model has run for a specified number of timesteps, the updated $\mathbf{H}^{(2)}[t]$ is passed to the next layer.

Proposed fGRU module A single-layer hGRU model efficiently solves a toy contour detection task by learning horizontal connections that implement a visual strategy of contour integration [28]. However, as we demonstrate in section 4, this model is suboptimal on the more challenging problem of neuron segmentation. Indeed, cognitive neuroscience suggests that top-down feedback from higher-processing regions is a key ingredient for perceptual grouping that is missing from this hGRU [37, 35]. Higher-level feature information can regulate and constrain the horizontal connections between units at a lower processing stage to efficiently group task-relevant image features [12]. A complementary perspective casts these low-level units as a recurrent “scratch-pad” for higher-level regions, which is directly read-out from to make decisions about the world after it is sufficiently processed by top-down feedback [13, 44]. Motivated by these considerations, we introduce a generalized expression of the hGRU module, the feedback gated recurrent (fGRU), which utilizes the dynamics of the hGRU in Eq. 2 to construct a model with both horizontal and top-down feedback connections.

The hGRU takes two inputs on every step of processing, $\mathbf{X}$ and $\mathbf{H}^{(2)}[t-1]$, corresponding to a feedforward drive from a prior layer, and the persistent output activity of the hGRU. An fGRU module can implement top-down feedback by relaxing constraints on the origins of these activities.

$$\mathbf{H}^{(2,l)}[t] = \text{fGRU}(\mathbf{H}^{(2,l)}[t], \mathbf{H}^{(2,l+1)}[t]) \quad (3)$$

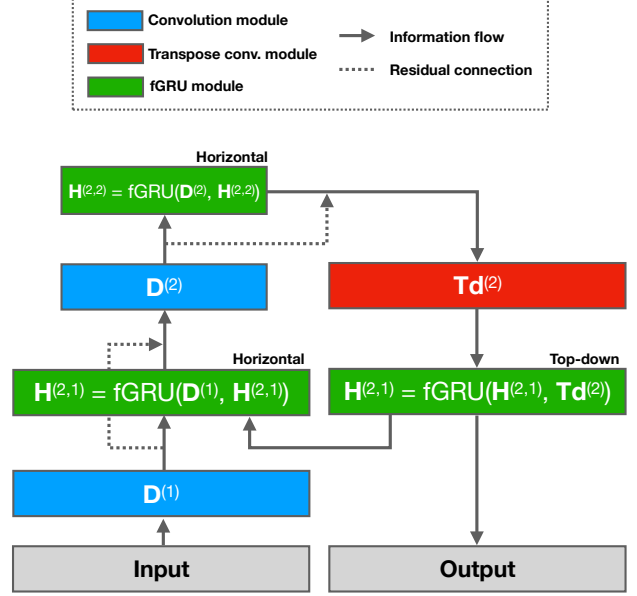


Figure 3. A high-level diagram of one timestep of processing in a two-layer fGRU (fGRU-2). Information flows serially through each layer of the model on every timestep, implementing a bottom-up and then top-down pass of activity. Blue rectangles depict convolutional modules (D), each of which consists of a convolutional layer, batch-norm. Max pool subsampling is shown by a reduced width of the rectangle for the following layer. The fGRU-2 features a transpose convolutional module, shown with the red rectangle, that upsamples activity from the second fGRU to match the height and width of the first. The three fGRU modules, shown in green rectangles, control the low-level horizontal interactions, high-level horizontal interactions, and top-down feedback from high-to-low level fGRU modules (ordered from the input to the output). Residual connections are added to improve learning, and denoted with dotted lines.

The function fGRU applies the hGRU dynamics in Eq. 2 to two input activities (Fig. 3). The feedforward drive input $\mathbf{X}$ of an hGRU module is substituted with $\mathbf{H}^{(2,l)}[t]$ which is the recently updated output of the fGRU module at layer l . Likewise, the output activity $\mathbf{H}^{(2)}[t-1]$ of an hGRU module is replaced with $\mathbf{H}^{(2,l+1)}[t]$ which denotes the output of another fGRU module at the subsequent layer $l+1$. An fGRU receiving these inputs will therefore learn to apply feedback from the high-level output (layer $l+1$) onto units in its low-level (layer l). On every timestep of processing, the gain activity selects low-level feature channels for processing, and the mix integrates the resulting activity with high-level horizontal activities. Because the fGRU is a generalization of the hGRU for feedback, it is important to note that a special case application of the fGRU can learn horizontal connections: $\mathbf{H}^{(2,l)}[t] = \text{fGRU}(\mathbf{X}^l, \mathbf{H}^{(2,l)}[t])$.

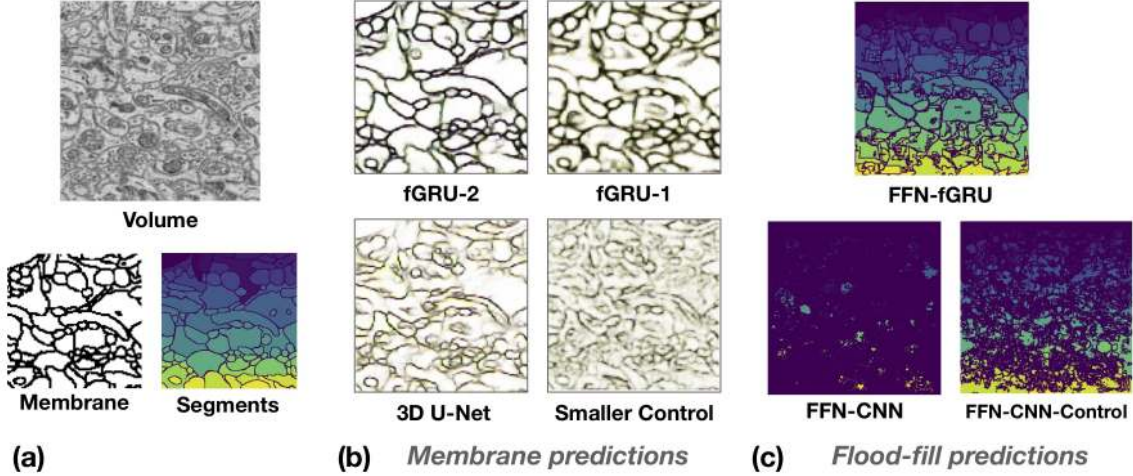


Figure 4. fGRU networks outperform a state-of-the-art 3D U-Net [25] for membrane detection on the STAR challenge. (a) An exemplar image, membrane labeling, and neurite segmentation from the STAR test volume. (b) Membrane predictions of fGRUs, a 3D U-Net [25], and a non-recurrent CNN with a similar number of parameters as the fGRU (Smaller Control). Model predictions are plotted as unthresholded nearest-neighbor affinity estimates in the X/Y/Z directions; higher-precision estimates result in high-contrast depictions of the membrane predictions. (c) Segmentation predictions from models based on flood-filling networks [18]. These are the reference FFN-CNN, a FFN-fGRU (an fGRU replacing its convolutional layers), and a non-recurrent FFN-CNN with a similar number of parameters as the FFN-fGRU (FFN-CNN-Control).

4. The STAR challenge

The STAR challenge introduces a labeled image stack of mouse retina from [2], which is $384 \times 384 \times 384$ voxels. Annotations are kept private for the challenge, and performance is evaluated on the private server at [STAR-challenge.github.io](https://star-challenge.github.io). The STAR training set consists of labeled volumes of fruit fly nervous system and mouse cortex. These datasets come from the MICCAI circuit reconstruction of electron microscopy images (CREMI), Segmentation of neurites in EM images (SNEMI3D; [20]), and FIB-25 (see Table 1).

Training procedure We trained two classes of models on the STAR challenge, which represent the leading approaches for automated connectome reconstruction. One approach is to predict the locations of cell membranes in a volume, and use postprocessing routines (such as watershed and GALA [33]) to derive segmentations. The other approach is to train a model to segment one neuron in a volume at a time [18]. The state of the art models of each approach are the 3D U-Net [25] and flood-filling networks (FFN-CNN [18]), respectively. Our implementations of these models are consistent with the published versions (see SI for details and validations). We also developed fGRU models and control models for each approach that were trained using identical training routines, which are discussed below. All models were trained using Tensorflow, and Adam [23].

Membrane prediction models Membrane prediction models were trained according to the routine described in [25]. Models were optimized for predicting the affinities of voxel pairs at a variety of distances, and a wide array of data augmentations were applied to the input volumes (only nearest-neighbor affinities were used for predictions; see SI for details). We focus on three membrane prediction models in addition to this reference 3D U-Net. First, we consider two models that use the fGRU module as an elementary component: (1) a one-layer fGRU model (fGRU-1) and (2) a two-layer fGRU (fGRU-2). We also consider a third model, a control CNN (FF-control) for understanding how the number of trainable parameters *per se* influences membrane prediction on the STAR challenge. Each model was trained for at least 100K steps using the learning rate schedule of [25]¹. Because the segmentation step in this approach is handled by hand-tuned post-processing routines, we only report membrane prediction performance here (but see Fig. S1 for segmentation performance of the reference 3D U-Net).

The fGRU-2 consists of two sets of convolutional layers followed by fGRU modules, with an intervening max pool subsampling using a $4 \times 4 \times 1$ kernel (corresponding to height, width, and depth) and 4×4 stride (Fig. 2). The convolutional layers both have $5 \times 5 \times 1$ kernels with 18 features. The kernel in the first fGRU module is $9 \times 9 \times 1$, the kernel in the second one is $5 \times 5 \times 3$, and both have

¹Another 3D U-Net was trained with L2 weight decay, but it was less effective on the STAR challenge than the reference model (mAP = 0.35).

	mAP	Parameters
FF-control	0.44	60K
3D U-Net [25]	0.51	1.4M
fGRU-1	0.60	45K
fGRU-2	0.72	50K

Table 2. Membrane prediction performance on the STAR challenge. Membrane predictions are evaluated by mean Average Precision (mAP; higher is better). For reference, a 3D U-Net [25] trained and tested on distinct sections of the STAR test volume scores 0.78 mAP.

18 features. In other words, these are kernels which learn horizontal connections in each layer, and only the second layer’s kernel learns these connections across three spatial dimensions. The fGRU modules also consist of separate sets of $1 \times 1 \times 1$ kernels $\mathbf{U}^{(1)}$, $\mathbf{U}^{(2)}$ with 18 features for calculating the gain and the mix in each fGRU module. A transpose convolution kernel with a $4 \times 4 \times 1$ kernel up-samples the second fGRU’s persistent activity to the height and width of its first fGRU. An additional fGRU module learns to implement top-down feedback from the second fGRU onto the first with a $1 \times 1 \times 1$ kernel, resulting in the operation: $\mathbf{H}^{(2,1)}[t] = \text{fGRU}(\mathbf{H}^{(2,1)}[t], \mathbf{H}^{(2,2)}[t])$. The fGRU-1 has a convolutional layer with $5 \times 5 \times 1$ kernels and 18 features, along with an fGRU for learning horizontal connections with a $9 \times 9 \times 3$ kernel. Both models are run for 8 timesteps, after which the persistent activity of their first fGRU ($\mathbf{H}^{(2,1)}$) is passed through batch-norm and a final convolutional classifier with a $1 \times 1 \times 1$ kernel to readout membrane predictions at every voxel.

Finally, as with the hGRU [28], the fGRU module weight kernel $\mathbf{W}$ is constrained to have symmetric weights between channels. This reduces the number of learnable parameters by nearly half vs. a typical convolutional kernel, and means that the weight $W_{x_0+\Delta x, y_0+\Delta y, k_1, k_2}$ is equal to the weight $W_{x_0+\Delta x, y_0+\Delta y, k_2, k_1}$, in which x_0 and y_0 denote the center of the kernel.

Membrane prediction performance We first performed a validation analysis to measure each model’s ability to predict membranes in the STAR test volume. Models were trained to predict voxel affinities on 80% of the STAR test volume images with single volume batches of size $160 \times 160 \times 18$ ($X \times Y \times Z$ dimensions) and tested on the remaining 20%. The fGRU-2 was on par with the 3D U-Net in predicting these membranes, and both of these models outpaced the others (see Table S1 in SI for results). Next, we used the same routine to train each model on samples from 80% of the STAR training dataset; 20% of it was held out for validation, which was used to select model weights for subsequent testing (Fig. 4a). When evaluated on the STAR test dataset, the 3D U-Net’s performance decreased by 0.27

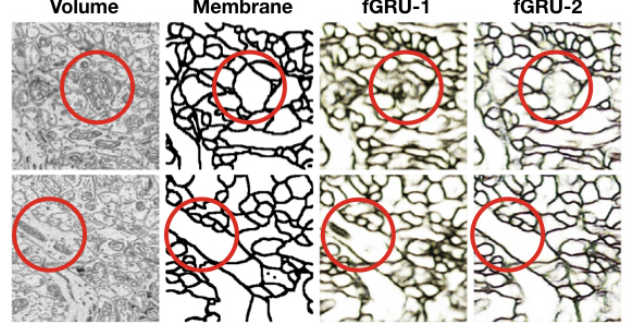


Figure 5. Top-down feedback gates the horizontal excitation and inhibition of low-level units. Exemplar images from the STAR test volume along with membrane labels and predictions from a 1- and 2-layer fGRU. Red circles demark regions in which the fGRU-1 shows uncontrolled excitation that spills into cell membranes. Top-down connections in the fGRU-2 correct this problem by using higher-level feature information to modulate low-level horizontal connections.

points of mean average precision (Table 2). This precipitous drop is evident in the model’s predictions: it confuses organelles in the SEM images with cell membranes, and fails to accurately group together continuous sections of cell membrane (Table 2 and Fig. 4b). By contrast, the fGRU-1 fairs significantly better and more consistently defines cell membranes.

The fGRU-2 further improves on the performance of the fGRU-1 (Table 2). Visual inspection of differences in predictions between these two models reveals that the fGRU-2 is more effective at suppressing irrelevant visual features within cells (Fig. 5). When learning horizontal connections, the fGRU-1 links together units in its feedforward drive that encode collinear features across long spatial distances, such as units that overlap an elongated membrane. A downside to this strategy is that it is unconstrained and can lead to excitation of task-irrelevant features. Top-down feedback in the fGRU-2 helps correct errors like “runaway” excitation by introducing high-level contextual information into the persistent activity of a low-level fGRU (red circles in Fig. 5 highlight examples of such inhibition).

One possible explanation for fGRU-based models outperforming the 3D U-Net on this challenge is that they contain only a fraction of the number of parameters (Table 2, Parameters). In other words, it is possible that parameter efficiency *per se* might predict performance on the STAR. To answer this question, we trained an additional feedforward model that replaced the fGRU modules in the fGRU-2 with 3D convolutional layers (kernel size: $7 \times 7 \times 3$), and removed both subsampling and upsampling, resulting in a similar number of parameters as the fGRU-2 (see SI for details²). This model was neither effective in the validation

²We trained another control with a similar number of parameters in a

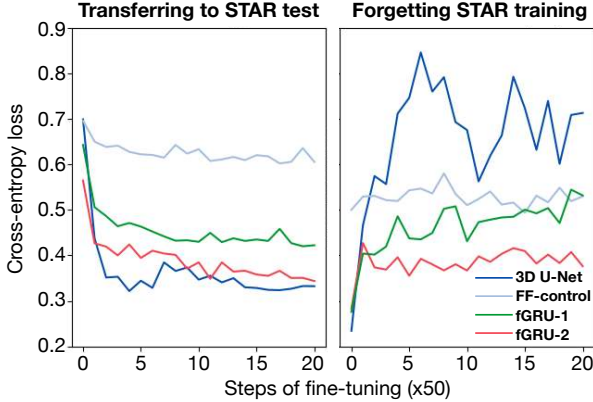


Figure 6. Forgetting correlates with model performance on the STAR challenge. Models are trained for the STAR challenge before their weights are “fine-tuned” to a cross-validation split (80/20) of the Briggman test volume. During fine-tuning, mean per-voxel crossentropy loss is recorded on the held-out split of the Briggman volume, as well as on a similar sized split from a volume in the STAR training dataset. All models become more effective at predicting membranes in the STAR test dataset from this transfer learning. fGRU models show less “forgetting” on a volume in the STAR training set (FIB-25), suggesting that they are less vulnerable to overfitting than the other models.

analysis of training and testing on the STAR test set (Table S1 in SI), nor did it perform well in the STAR challenge (Table 2 and Fig. 4b).

Is there a metric that predicts model performance? After training these models on the STAR challenge, we investigated their ability to learn the STAR test dataset via fine-tuning (Fig. 6, left panel). At the same time, we recorded model performance in predicting membranes on the STAR training dataset (Fig. 6, right panel). These performance curves indicate that the 3D U-Net is very amenable to fine-tuning, and rapidly learns to precisely predict membranes in the STAR test dataset. However, during fine-tuning, it completely “forgets” the original dataset. The fGRU-1 shows a similar pattern of results, but with less effective transfer and less severe forgetting. The fGRU-2, on the other hand, transfers nearly as quickly as the 3D U-Net while also maintaining information about the STAR training dataset. Together, these results suggest that a model’s tendency towards “catastrophic forgetting” [30] is a useful metric for measuring overfitting, and in turn, predicting its ability to generalize to the STAR test set.

Flood-filling models Flood-filling models were trained according to the routine described in [18] with slight variations to improve the speed of convergence (see SI for details). We trained three different versions of this model:

3D U-Net architecture, but it was less effective than this model.

	Precision	Recall	Parameters
FFN-CNN [18]	0.772	0.057	630K
FFN-CNN-Control	0.821	0.063	196K
FFN-fGRU	0.917	0.414	126K

Table 3. Segmentation performance on the STAR challenge, evaluated by “patchwise” precision and recall (higher is better). These metrics are computed as the mean precision and recall of predicted cell masks over 500 iterations of network processing in randomly sampled fields of vision (hence, “patchwise”). The final segmentation produced by flood-filling networks depends on hand-tuned hyperparameters. Patchwise precision-recall better reflects the quality of the models’ cell segmentations because it does not depend on such hand-tuning.

(i) the publicly available reference version from [18] (FFN-CNN); (ii) a version with an fGRU in place of its convolutional network; (iii) a “shallower” variant of the FFN-CNN to study the effect of reduced feature depth and overall network size (FFN-CNN-Control). See SI for details on model architectures and training routines.

Flood-filling performance These models were first validated on their ability to segment neurons in the STAR test set when trained on a held-out section of the same volume (see Table S2 in SI for results): FFN-CNN and FFN-fGRU performed similarly, and both outperformed the FFN-CNN-Control. We next trained these models on the STAR challenge, and then tested them on the STAR test volume (Table 3). The FFN-CNN did not generalize to this volume. Simply using a smaller and shallower network did not rescue performance either: the FFN-CNN-Control was also poor at segmenting this test volume. The FFN-fGRU, on the other hand, produced sensible, albeit oversegmented, predictions (Fig. 4).

5. Discussion

We have presented STAR, a novel connectomics challenge, aimed at testing the ability of architectures for neuron reconstruction to generalize to novel brain tissues beyond those used for training. We have shown that state-of-the-art architectures perform well when trained and tested on different subsets of the same volume but generalize poorly to different volumes. That convolutional neural networks (CNNs) exhibit a limited ability to generalize beyond training data is known [1, 56, 19, 10, 52, 42, 39] but often overlooked. Our results are consistent with a recent study that demonstrated that CNNs trained for image categorization on CIFAR do not generalize well to images from a novel test set even though the training and test sets had near identical statistics [36]. Similarly, it has been shown that while CNNs can be trained to handle specific kinds of noise, they are unable to generalize to unseen (albeit similar) noise con-

ditions [10].

We have further described a novel deep learning module, the feedback gated recurrent unit (fGRU), which leverages recurrent interactions – via both short-range horizontal connections within a convolution layer and long-range top-down connections between layers. The architecture was shown to generalize better than all tested baselines. Our work further adds to a growing body of literature which suggests that, consistent with the anatomy and physiology of the visual cortex, feedback connections may be critical to visual recognition tasks. Previous work has shown how feedback may provide prior information in predictive coding models [16, 29, 34] and help solve contour tracing, incremental grouping, object recognition performance [11, 41, 45, 49, 32], and other visual recognition tasks that require learning long-range statistical dependencies [3, 28]. The present study also demonstrates the limitations of the horizontal interactions implemented by one of these models, the hGRU [28], which are strained by the visual complexity of neuron segmentation in SEM images. The fGRU corrects this deficiency by controlling horizontal connections at a lower layer with top-down feedback connections from a higher one, which we find helps to suppress some of the pathological dynamics that emerge in a single hGRU layer. One possible functional explanation for this is that the fGRU disambiguates multiple interpretations of the visual image at a lower layer by introducing contextual information from a higher layer.

In summary, this work advocates for shifting the emphasis of computer vision in connectomics from fitting volume-specific datasets to generalizing across volumes. It is our hope that the STAR challenge and fGRU model framework constitute building blocks towards this goal.

5.1. Acknowledgements

We are grateful to the authors of [18] who have helped us implement their model and Kalpit Thakkar who created the challenge website. This research was supported by the Carney Institute for Brain Science through an innovation award granted to TS and DB. Additional support was provided by the Brown University Center for Computation and Visualization (CCV), as well as Meridian IT and IBM.

References

- [1] A. Azulay and Y. Weiss. Why do deep convolutional networks generalize so poorly to small image transformations? *arXiv preprint arXiv:1805.12177 [cs.CV]*, May 2018.
- [2] K. L. Briggman, M. Helmstaedter, and W. Denk. Wiring specificity in the direction-selectivity circuit of the retina. *Nature*, 471:183, Mar. 2011.
- [3] T. Brosch, H. Neumann, and P. R. Roelfsema. Reinforcement learning of linking and tracing contours in recurrent neural networks. *PLoS Computational Biology*, 11(10):e1004489, Oct. 2015.
- [4] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using RNN Encoder–Decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, Stroudsburg, PA, USA, 2014. Association for Computational Linguistics.
- [5] T. Cooijmans, N. Ballas, C. Laurent, Ç. Gülçehre, and A. Courville. Recurrent batch normalization. *arXiv preprint arXiv:1603.09025 [cs.LG]*, Mar. 2016.
- [6] W. Denk and H. Horstmann. Serial block-face scanning electron microscopy to reconstruct three-dimensional tissue nanostructure. *PLoS Biology*, 2(11):e329, Nov. 2004.
- [7] H. Ding, R. G. Smith, A. Poleg-Polsky, J. S. Diamond, and K. L. Briggman. Species-specific wiring for direction selectivity in the mammalian retina. *Nature*, 535(7610):105–110, July 2016.
- [8] A. L. Eberle, S. Mikula, R. Schalek, J. Lichtman, M. L. Knothe Tate, and D. Zeidler. High-resolution, high-throughput imaging with a multibeam scanning electron microscope. *Journal of microscopy*, 259(2):114–120, Aug. 2015.
- [9] A. Fornito, A. Zalesky, and M. Breakspear. The connectomics of brain disorders. *Nature Reviews Neuroscience*, 16:159, Feb. 2015.
- [10] R. Geirhos, D. H. J. Janssen, H. H. Schütt, J. Rauber, M. Bethge, and F. A. Wichmann. Comparing deep neural networks against humans: object recognition when the signal gets weaker. *arXiv preprint arxiv:1706.06969 [cs.CV]*, June 2017.
- [11] D. George, W. Lehrach, K. Kinsky, M. Lázaro-Gredilla, C. Laan, B. Marthi, X. Lou, Z. Meng, Y. Liu, H. Wang, A. Lavin, and D. S. Phoenix. A generative vision model that trains with high data efficiency and breaks text-based CAPTCHAs. *Science*, 358(6368), Dec. 2017.
- [12] C. D. Gilbert and W. Li. Top-down influences on visual processing. *Nature Reviews Neuroscience*, 14(5):350–363, May 2013.
- [13] C. D. Gilbert and M. Sigman. Brain states: top-down influences in sensory processing. *Neuron*, 54(5):677–696, June 2007.
- [14] S. Grossberg and E. Mingolla. Neural dynamics of perceptual grouping: textures, boundaries, and emer-

- gent segmentations. *Perception & Psychophysics*, 38(2):141–171, Aug. 1985.
- [15] D. Haehn, V. Kaynig, J. Tompkin, J. W. Lichtman, and H. Pfister. Guided proofreading of automatic segmentations for connectomics. *arXiv preprint arXiv:1704.00848 [cs.CV]*, Apr. 2017.
 - [16] K. Han, H. Wen, Y. Zhang, D. Fu, E. Culurciello, and Z. Liu. Deep predictive coding network with local recurrent processing for object recognition. *arXiv preprint arXiv:1805.07526 [cs.CV]*, May 2018.
 - [17] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. *arXiv preprint arXiv:1512.03385 [cs.CV]*, Dec. 2015.
 - [18] M. Januszewski, J. Kornfeld, P. H. Li, A. Pope, T. Blakely, L. Lindsey, J. Maitin-Shepard, M. Tyka, W. Denk, and V. Jain. High-precision automated reconstruction of neurons with flood-filling networks. *Nature Methods*, 15(8):605–610, Aug. 2018.
 - [19] J. Jo and Y. Bengio. Measuring the tendency of CNNs to learn surface statistical regularities. *arXiv preprint arXiv:1711.11561 [cs.LG]*, Nov. 2017.
 - [20] N. Kasthuri, K. J. Hayworth, D. R. Berger, R. L. Schalek, J. A. Conchello, S. Knowles-Barley, D. Lee, A. Vázquez-Reina, V. Kaynig, T. R. Jones, M. Roberts, J. L. Morgan, J. C. Tapia, H. S. Seung, W. G. Roncal, J. T. Vogelstein, R. Burns, D. L. Sussman, C. E. Priebe, H. Pfister, and J. W. Lichtman. Saturated reconstruction of a volume of neocortex. *Cell*, 162(3):648–661, July 2015.
 - [21] J. Kim, J. K. Lee, and K. M. Lee. Deeply-Recursive convolutional network for image Super-Resolution. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1637–1645. IEEE, June 2016.
 - [22] J. S. Kim, M. J. Greene, A. Zlateski, K. Lee, M. Richardson, S. C. Turaga, M. Purcaro, M. Balkam, A. Robinson, B. F. Behabadi, M. Campos, W. Denk, H. S. Seung, and EyeWriters. Space-time wiring specificity supports direction selectivity in the retina. *Nature*, 509(7500):331–336, May 2014.
 - [23] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980 [cs.LG]*, Dec. 2014.
 - [24] C. Kizilyaprak, J. Daraspe, and B. M. Humbel. Focused ion beam scanning electron microscopy in biology. *Journal of Microscopy*, 254(3):109–114, June 2014.
 - [25] K. Lee, J. Zung, P. Li, V. Jain, and H. Sebastian Seung. Superhuman accuracy on the SNEMI3D connectomics challenge. *arXiv preprint arXiv:1706.00120 [cs.CV]*, May 2017.
 - [26] M. Liang and X. Hu. Recurrent convolutional neural network for object recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3367–3375, 2015.
 - [27] Q. Liao and T. Poggio. Bridging the gaps between residual learning, recurrent neural networks and visual cortex. *arXiv preprint arXiv:1604.03640 [cs.LG]*, Apr. 2016.
 - [28] D. Linsley, J. Kim, V. Veerabadran, and T. Serre. Learning long-range spatial dependencies with horizontal gated-recurrent units. *arXiv preprint arXiv:1805.08315 [cs.CV]*, May 2018.
 - [29] W. Lotter, G. Kreiman, and D. Cox. Deep predictive coding networks for video prediction and unsupervised learning. *arXiv preprint arXiv:1605.08104 [cs.LG]*, May 2016.
 - [30] N. Y. Masse, G. D. Grant, and D. J. Freedman. Alleviating catastrophic forgetting using context-dependent gating and synaptic stabilization. *Proceedings of the National Academy of Sciences of the United States of America (PNAS)*, 115(44):E10467–E10475, Oct. 2018.
 - [31] D. A. Mély, D. Linsley, and T. Serre. Complementary surrounds explain diverse contextual phenomena across visual modalities. *Psychological Review*, 125(5):769–784, Oct. 2018.
 - [32] A. Nayebi, D. Bear, J. Kubilius, K. Kar, S. Ganguli, D. Sussillo, J. J. DiCarlo, and D. L. K. Yamins. Task-Driven convolutional recurrent models of the visual system. *arXiv preprint arXiv:1807.00053 [q-bio.NC]*, June 2018.
 - [33] J. Nunez-Iglesias, R. Kennedy, T. Parag, J. Shi, and D. B. Chklovskii. Machine learning of hierarchical clustering to segment 2D and 3D images. *PloS One*, 8(8):e71715, Aug. 2013.
 - [34] R. C. O’Reilly, D. R. Wyatte, and J. Rohrlich. Deep predictive learning: A comprehensive model of three visual streams. *arXiv preprint arXiv:1709.04654 [q-bio.NC]*, Sept. 2017.
 - [35] P. Parent and S. W. Zucker. Trace inference, curvature consistency, and curve detection. *IEEE Trans. Pattern Anal. Mach. Intell.*, 11(8):823–839, Aug. 1989.
 - [36] B. Recht, R. Roelofs, L. Schmidt, and V. Shankar. Do CIFAR-10 classifiers generalize to CIFAR-10? *arXiv preprint arXiv:1806.00451 [cs.LG]*, June 2018.
 - [37] P. R. Roelfsema and F. P. de Lange. Early visual cortex as a multiscale cognitive blackboard. *Annual Review of Vision Science*, 2:131–151, Oct. 2016.
 - [38] O. Ronneberger, P. Fischer, and T. Brox. U-Net: Convolutional networks for biomedical image segmenta-

- tion. *arXiv preprint arXiv:1505.04597 [cs.CV]*, May 2015.
- [39] A. Rosenfeld, R. Zemel, and J. K. Tsotsos. The elephant in the room. *arXiv preprint arXiv:1808.03305 [cs.CV]*, Aug. 2018.
- [40] S. Sabbah, D. Berg, C. Papendorp, K. L. Briggman, and D. M. Berson. A cre mouse line for probing irradiance- and Direction-Encoding retinal networks. *eNeuro*, 4(2), Mar. 2017.
- [41] S. Sabour, N. Frosst, and G. E. Hinton. Dynamic routing between capsules. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems (NIPS) 30*, pages 3856–3866, 2017.
- [42] B. Saleh, A. M. Elgammal, J. Feldman, and A. Farhadi. Toward a taxonomy and computational models of abnormalities in images. In *AAAI*, pages 3588–3596, 2016.
- [43] M. Schröter, O. Paulsen, and E. T. Bullmore. Micro-connectomics: probing the organization of neuronal networks at the cellular scale. *Nature Reviews Neuroscience*, 18:131, Feb. 2017.
- [44] S. D. Slotnick, W. L. Thompson, and S. M. Kosslyn. Visual mental imagery induces retinotopically organized activation of early visual areas. *Cerebral Cortex*, 15(10):1570–1583, Oct. 2005.
- [45] C. J. Sporer, P. McClure, and N. Kriegeskorte. Recurrent convolutional neural networks: A better model of biological object recognition. *Frontiers in Psychology*, 8:1551, Sept. 2017.
- [46] J. K. Stevens, T. L. Davis, N. Friedman, and P. Sterling. A systematic approach to reconstructing microcircuitry by electron microscopy of serial sections. *Brain Research*, 2(3):265–293, Dec. 1980.
- [47] L. W. Swanson and J. W. Lichtman. From cajal to connectome and beyond. *Annual Reviews Neuroscience*, 39(1):197–216, July 2016.
- [48] S.-Y. Takemura, A. Bharioke, Z. Lu, A. Nern, S. Vitaladevuni, P. K. Rivlin, W. T. Katz, D. J. Olbris, S. M. Plaza, P. Winston, T. Zhao, J. A. Horne, R. D. Fetter, S. Takemura, K. Blazek, L.-A. Chang, O. Ogundeyi, M. A. Saunders, V. Shapiro, C. Sigmund, G. M. Rubin, L. K. Scheffer, I. A. Meinertzhagen, and D. B. Chklovskii. A visual motion detection circuit suggested by drosophila connectomics. *Nature*, 500(7461):175–181, Aug. 2013.
- [49] H. Tang, M. Schrimpf, W. Lotter, C. Moerman, A. Paredes, J. Ortega Caro, W. Hardesty, D. Cox, and G. Kreiman. Recurrent computations for visual pattern completion. *Proceedings of the National Academy of Sciences of the United States of America (PNAS)*, Aug. 2018.
- [50] M. P. van den Heuvel, E. T. Bullmore, and O. Sporns. Comparative connectomics. *Trends in Cognitive Sciences*, 20(5):345–361, May 2016.
- [51] A. Vishwanathan, K. Daie, A. D. Ramirez, J. W. Lichtman, E. R. F. Aksay, and H. S. Seung. Electron microscopic reconstruction of functionally identified cells in a neural integrator. *Current Biology*, 27(14):2137–2147.e3, July 2017.
- [52] J. Wang, Z. Zhang, C. Xie, Y. Zhou, V. Premachandran, J. Zhu, L. Xie, and A. Yuille. Visual concepts and compositional voting. *arXiv preprint arXiv:1711.04451 [cs.CV]*, Nov. 2017.
- [53] J. G. White, E. Southgate, J. N. Thomson, and S. Brenner. The structure of the nervous system of the nematode *caenorhabditis elegans*. *Philosophical Transactions of the Royal Society of London. Series B, Biological sciences*, 314(1165):1–340, Nov. 1986.
- [54] R. Xiao Feng and L. Bo. Discriminatively trained sparse code gradients for contour detection. In F. Pereira, C. J. C. Burges, L. Bottou, and K. Q. Weinberger, editors, *Advances in Neural Information Processing Systems (NIPS) 25*, pages 584–592. Curran Associates, Inc., 2012.
- [55] A. R. Zamir, T.-L. Wu, L. Sun, W. Shen, B. E. Shi, J. Malik, and S. Savarese. Feedback Networks. *arXiv preprint arxiv: 1612.09508*, Dec. 2016.
- [56] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals. Understanding deep learning requires rethinking generalization. *arXiv preprint arXiv:1611.03530 [cs.LG]*, Nov. 2016.

Supplementary Information: Robust neural circuit reconstruction from serial electron microscopy with convolutional recurrent networks

6. Membrane prediction models

Training and testing Our reference model for membrane prediction is the 3D U-Net of [25]. We followed their published routine for training all membrane prediction models. Key to this is their use of a large set of random data augmentations applied to SEM image volumes, which simulate common noise and errors in SEM imaging. These are (i) misalignment between consecutive z -locations in each input image volume. (ii) Partial- or fully-missing sections of the input image volumes. (iii) Blurring of portions of the image volume. Augmentations that simulated these types of noise, as well as random flips over the xyz -plane, rotations by 90° , brightness and contrast perturbations, were applied to volumes following the settings of [25]. Models were trained to convergence, for at least 100K iterations, using Adam [23] and the learning rate schedule of [25], in which the optimizer step-size was halved when validation loss stopped decreasing (up to four times). Models were trained with single-SEM volume batches of $160 \times 160 \times 18$ ($X/Y/Z$), normalized to $[0, 1]$. All models were trained to predict nearest-neighbor voxel affinities, as well as 3 other mid- to long-range voxel distances, as in [25]. Only the nearest neighbors were used at test time.

We performed 3D neuron reconstructions on membrane predictions using a pipeline consisting of watershed to derive supervoxels and GALA [33] for agglomerating these into precise segmentations. We used this approach to validate our implementation of the 3D U-Net [25] (Fig. S1).

We validated membrane prediction models on the STAR challenge test set. Models were trained on the first 80% of the volume’s z -axis images, and tested on the remainder. Results are shown as mean average precision (mAP; scores calculated per z -axis image and then averaged together) in Table S1.

fGRU membrane prediction After training fGRU-based models on the STAR challenge, we investigated how the top-down feedback of the fGRU-2 influenced its predictions. Exemplar slices from the STAR test dataset are depicted in Fig. S2, along with the ground-truth membrane labels, and per-pixel membrane predictions from a 1- and 2-layer fGRU. The most apparent differences between the two models is that the top-down feedback of the 2-layer fGRU has stronger control over the incorrect extension of membrane predictions into cell parts. These results, which depict nearest-neighbor affinity predictions, also reveal that

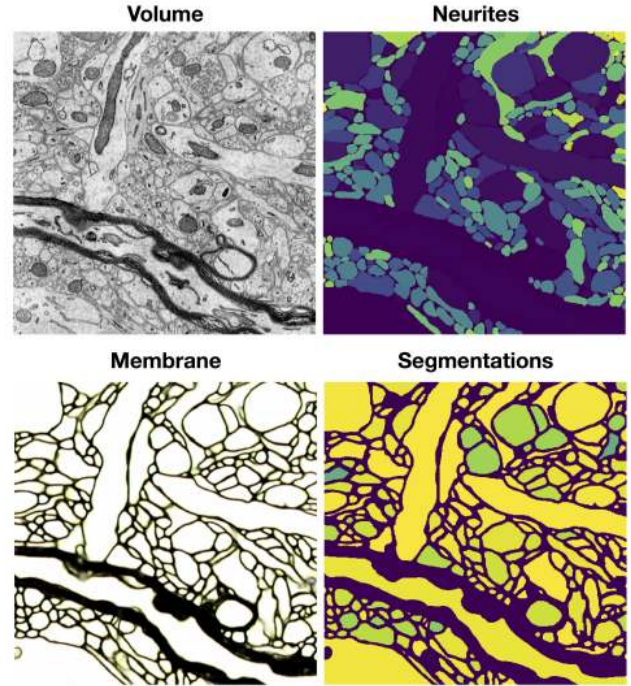


Figure S1. We trained the reference 3D U-Net from [25] on the SNEMI3D dataset to validate the implementation. Segmentations here are derived by watershed and agglomeration with GALA [33], resulting in “superhuman” ARAND (evaluated according to the SNEMI3D standard; lower is better) of 0.04, which is below the reported human-performance threshold of 0.06 and on par with the published result (see Table 1 in [25], mean affinity agglomeration).

the fGRU-2 was more confident in its membrane predictions than the fGRU-1.

7. Flood-filling models

The fGRU* module We discovered that several modifications to the original fGRU module led to significantly better generalization performance when deriving neuron segmentations with the flood-filling network approach (Fig. S3). These modifications result in a module called fGRU*, which makes the following changes to the fGRU: (i) it processes horizontal (i.e., persistent activity at layer l) and top-down interactions (i.e., persistent activity at layer $l + 1$) within a single module. By contrast, the fGRU discussed in the main text only processes one of these interactions at a time. (ii) Inspired by [12], the module uses top-

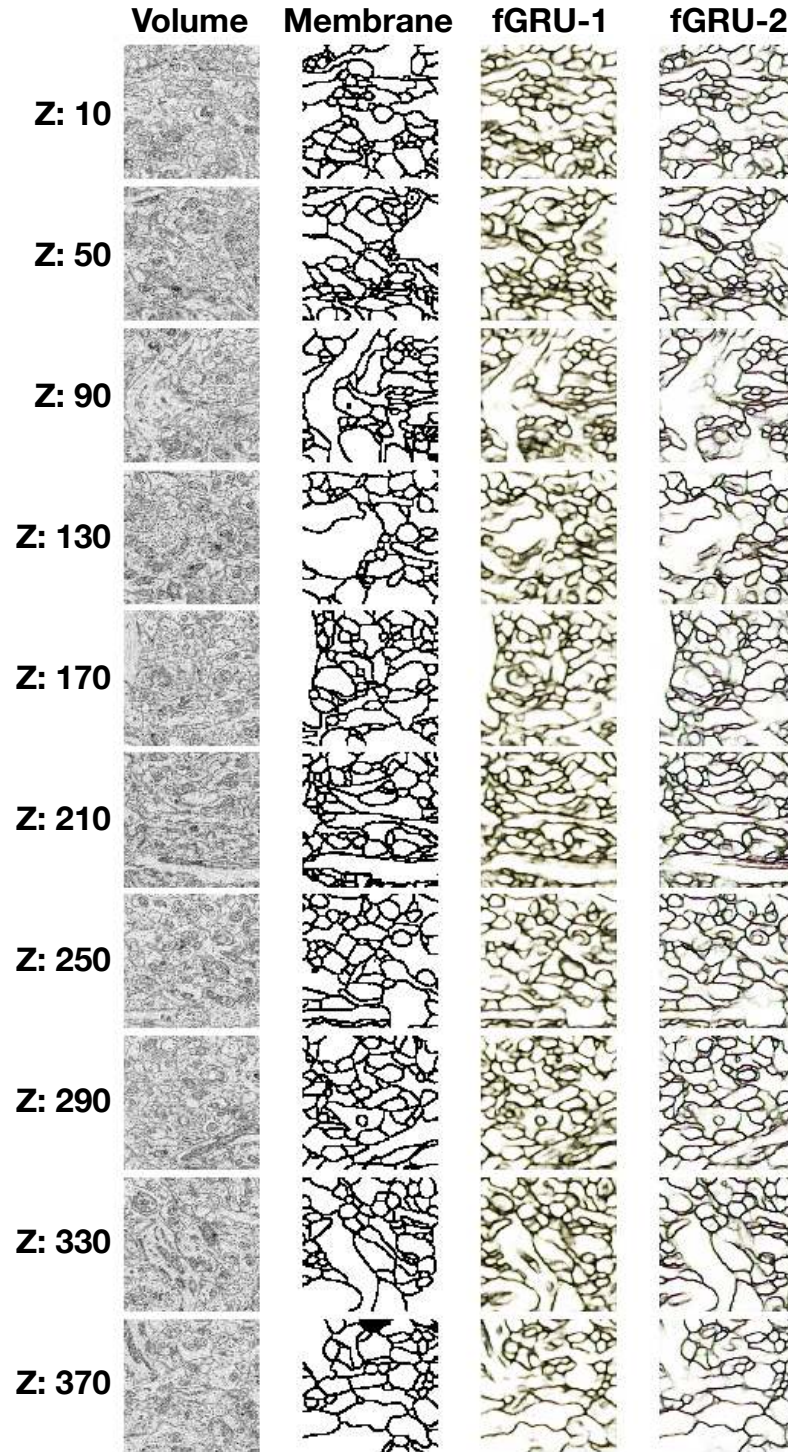


Figure S2. An extended comparison of membrane prediction performance for the one- and two-layer fGRU models (fGRU-1 and fGRU-2) on images from the STAR test dataset. Note the increased sparsity of the fGRU-2 predictions vs. the fGRU-1. The higher contrast of the fGRU-2 predictions indicates greater confidence than the fGRU-1.

down feedback to compute its gain and mix activities. (iii) The module includes a separate gain control activity to se-

lectively modulate the flow of the feedforward drive $\mathbf{X}$ into its hidden state $\mathbf{H}^{(2)}$. This additional gain $\mathbf{G}^{(1b)}$ joins the

	mAP	Parameters
FF-Control	0.63	60K
fGRU-1	0.74	45K
fGRU-2	0.77	50K
3D U-Net [25]	0.78	1.4M

Table S1. Reference performance for models after training and testing on distinct image splits (80%/20%) of the STAR test volume. Membrane predictions are evaluated by mean Average Precision (mAP; higher is better).

existing gain $\mathbf{G}^{(1a)}$ (which is applied to the module’s persistent activity). We describe the fGRU* with the following equations:

$$\begin{aligned}
\mathbf{G}^{(1a)}[t] &= \sigma(\text{BN}(\mathbf{U}^{(1a)} * (\kappa_1(\mathbf{H}^{(2)}[t-1] \odot \mathbf{Td}) + \kappa_2\mathbf{H}^{(2)}[t-1] + \kappa_3\mathbf{Td}))) \\
\tilde{\mathbf{X}} &= \mathbf{X} \odot \mathbf{G}^{(1a)}[t] \\
\mathbf{G}^{(1b)}[t] &= \sigma(\text{BN}(\mathbf{U}^{(1b)} * (\eta_1(\tilde{\mathbf{X}} \odot \mathbf{Td}) + \eta_2\tilde{\mathbf{X}} + \eta_3\mathbf{Td}))) \\
\mathbf{C}^{(1)}[t] &= \text{BN}(\mathbf{W} * (\mathbf{G}^{(1b)}[t] \odot \mathbf{H}^{(2)}[t-1])) \\
\mathbf{G}^{(2)}[t] &= \sigma(\text{BN}(\mathbf{U}^{(2)} * (\lambda_1(\mathbf{H}^{(1)}[t] \odot \mathbf{Td}) + \lambda_2\mathbf{H}^{(1)}[t] + \lambda_3\mathbf{Td})))) \\
\mathbf{C}^{(2)}[t] &= \text{BN}(\mathbf{W} * \mathbf{H}^{(1)}[t])
\end{aligned} \tag{4}$$

The activities $\mathbf{H}^{(1)}[t]$, $\tilde{\mathbf{H}}^{(2)}[t]$ and $\mathbf{H}^{(2)}[t]$ are calculated identically to the fGRU described in the main text. In the fGRU*, the gains $\mathbf{G}^{(1a)}$ and $\mathbf{G}^{(1b)}$, and the mix $\mathbf{G}^{(2)}$ are computed via $1 \times 1 \times 1$ convolution using kernels $\mathbf{U}^{(1a)}$, $\mathbf{U}^{(1b)}$ and $\mathbf{U}^{(2)}$, respectively. These activities are computed with a combination of top-down feedback (i.e., , persistent activity from layer $l+1$; $\mathbf{Td}$) with input or horizontal activity (i.e., , $\mathbf{X}$ or persistent activity from layer l ; $\mathbf{H}^{(2)}$). The calculation of these activities also includes additional degrees of freedom vs. the fGRU of the main text. A learned scalar is included in the calculation of each to balance between linear (additive) and quadratic (multiplicative) combinations of horizontal and top-down activity: (i) linear and quadratic contributions to calculation of $\mathbf{G}^{(1a)}$ are modulated by κ ; (ii) for $\mathbf{G}^{(1b)}$, η has the same function; and (iii) for the mix, λ performs this role.

The FFN-fGRU network We constructed a three-layer architecture, called the FFN-fGRU, which we pitted against the FFN-CNN in segmenting neurons in the STAR challenge (Fig. 4 in the main text and Fig. S4, here). This network uses three downsampling and three upsampling steps to combine hierarchical feature processing with horizontal and top-down feedback recurrence at every layer, im-

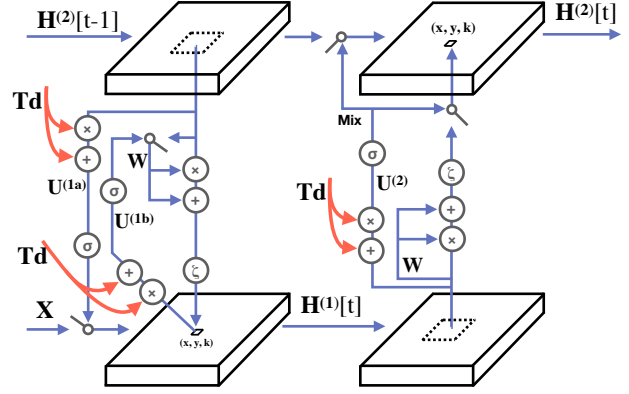


Figure S3. A diagram of the fGRU* module used in the FFN-fGRU. This module uses top-down feedback ($\mathbf{Td}$) to compute its gains, $\mathbf{G}^{(1a)}$, $\mathbf{G}^{(1b)}$, and $\mathbf{G}^{(2)}$. In this module, $\mathbf{Td}$ helps to gate horizontal interactions implemented by the kernel $\mathbf{W}$. In addition, we found that introducing an extra gate that controls the flow of $\mathbf{X}$ improved performance in neuron segmentation. This allows the network to selectively modulate its feedforward drive while calculating updates to its hidden state $\mathbf{H}^{(2)}$.

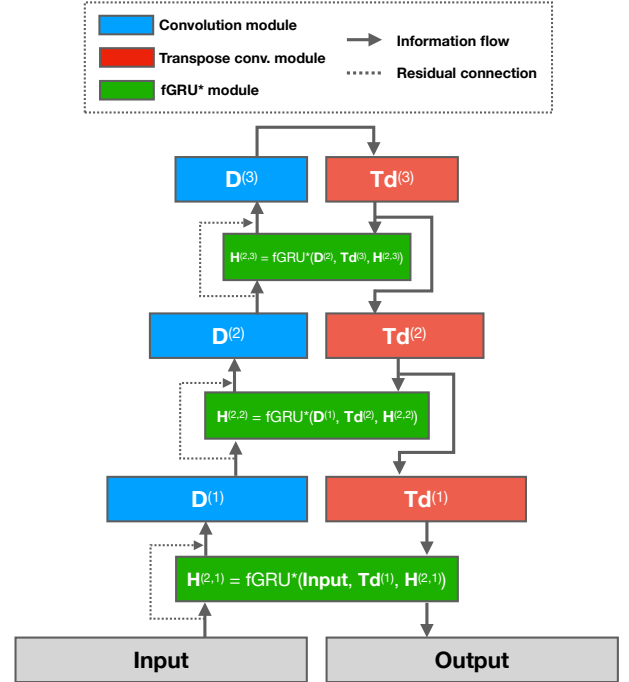


Figure S4. A high-level diagram of a FFN-fGRU. Three fGRU* modules are interleaved by three convolutional and three transpose convolutional modules.

plemented by fGRU* modules³. The shape of horizontal kernels used in the FFN-fGRU model’s fGRU* modules (in

³We found that increasing model depth led to a significant improvement in performance on this flood-filling segmentation task.

	Precision	Recall	Parameters
FFN-CNN [18]	0.988	0.772	630K
FFN-CNN-Ctrl	0.951	0.657	196K
FFN-fGRU	0.994	0.853	126K

Table S2. Reference performance for models after training and testing on distinct image splits (50%/50%) within the STAR test volume, evaluated by “patchwise” precision and recall (higher is better). These metrics are computed as the mean precision and recall of predicted cell masks over 500 iterations of network processing in randomly sampled fields of vision (hence, “patchwise”).

height $\times$ width $\times$ depth) are $7 \times 7 \times 1$, $5 \times 5 \times 3$ and $3 \times 3 \times 3$, respectively. Each convolutional module is a combination of three steps: (1) A combination step which mixes the output of the preceding fGRU module ($\mathbf{H}^{(2,l)}$) with a residual bypass ($\mathbf{X}$ or $\mathbf{D}^{(l-1)}$) from the previous layer (described below); (2) 3-D convolution; and (3) 3-D max pooling and subsampling (Fig. S4).

The combination step of (1) is similar to gain and mix computation in the fGRU* module. This step combines three terms $\mathbf{H}^{(2,l)}$, $\mathbf{D}^{(l-1)}$, and $\mathbf{H}^{(2,l)} \odot \mathbf{D}^{(l-1)}$ – using a trainable scalar weight. The shape of convolution kernels used in the first, second and third convolution modules are $7 \times 7 \times 1$, $5 \times 5 \times 1$, and $5 \times 5 \times 1$, respectively (height/width/depth). The shape of the pooling kernels are $2 \times 2 \times 1$, $2 \times 2 \times 2$, and $2 \times 2 \times 1$, respectively. Subsampling at every stage is performed with the same rate specified by the pooling kernels. Each transpose convolution module uses strided transpose convolution to upsample a high-level feature map to match the shape of a low-level feature map. The stride in each transpose convolutional module is the same as the downsampling rate of the corresponding convolutional module. The shape of convolution kernels in the first, second and third convolution modules are $8 \times 8 \times 1$, $6 \times 6 \times 2$, and $6 \times 6 \times 1$, respectively. The FFN-fGRU network ran for 8 timesteps for every iteration of flood-filling. Like the fGRU discussed in the main text, fGRU* used symmetric weight sharing to reduce its number of free parameters (see Section 6 in the main text).

Training and testing Flood-filling networks were trained according to the routines of [18]. The reference implementation of the FFN-CNN was taken from a public repository of those authors (<https://www.github.com/google/ffn>). We made two changes to the original configuration that accelerated and improved training. First, the shape of the network’s field of view (FOV) changed from the original $33 \times 33 \times 17$ voxels to $57 \times 57 \times 13$ as it improved the rate of convergence for the FFN-fGRU. We tried this approach with the FFN-CNNs as well; however, we found that its training was more effective with the original

field of view, and therefore report results from this configuration. Second, networks were trained with Adam ($1e^{-3}$ learning rate) instead of stochastic gradient descent because it led to a faster convergence. All networks were trained with batches of 16 volumes. Training stopped when validation accuracy, measured in F1 score over a batch of 500 FOVs, stopped improving by more than 3% over 100K iterations. Validation was performed every 25K training iterations. According to this rule, the FFN-CNN was trained for 475K iterations, the FFN-CNN-Control for 550K iterations, and the FFN-fGRU for 375K iterations. Models were trained on the STAR challenge (see main text), and on a split half of the STAR test set for validation (Table. S2).

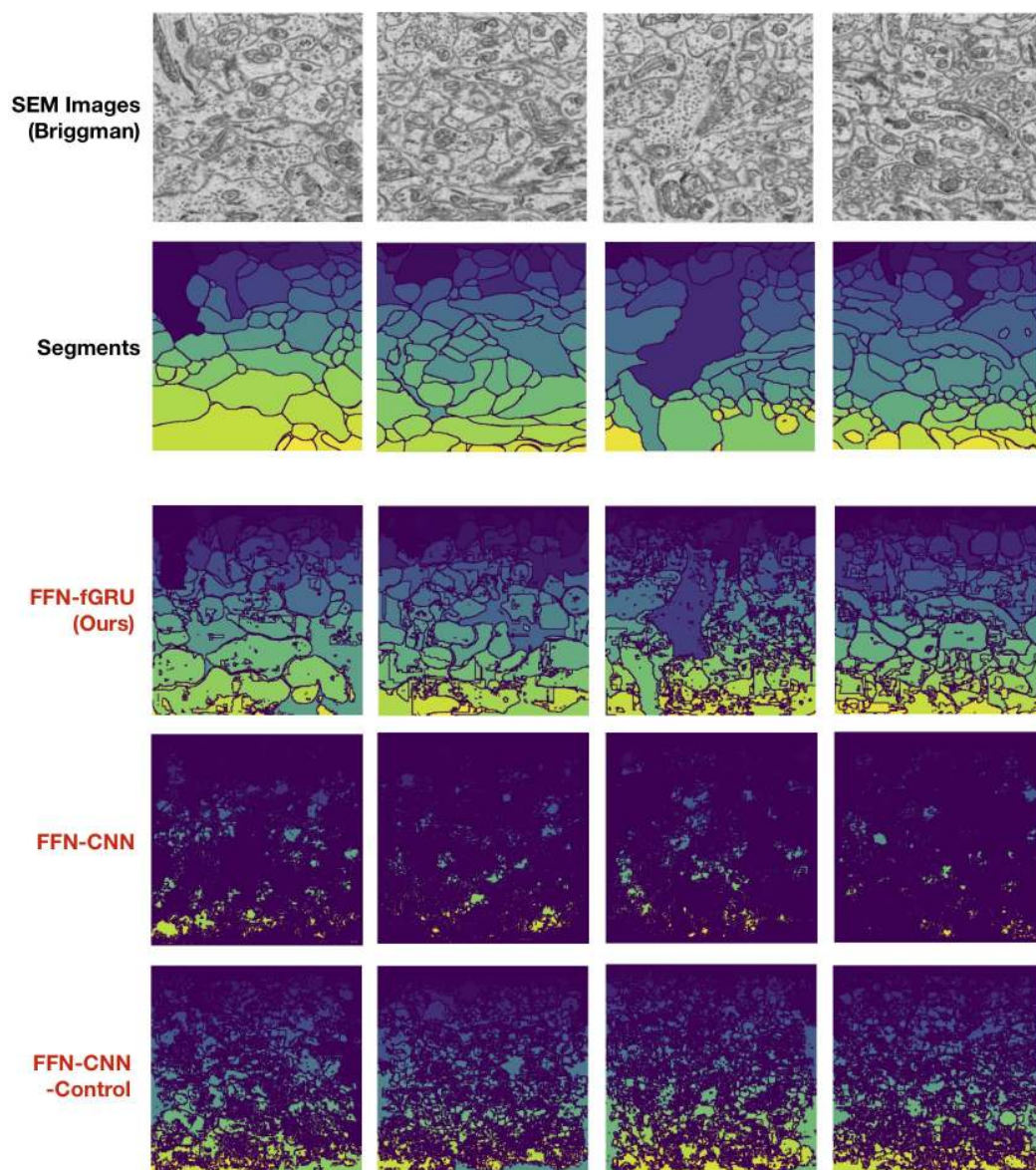


Figure S5. Exemplar SEM images from the STAR test dataset (Briggman), their ground-truth neuron annotations, and predicted segmentations from FFN models after training on the STAR train dataset.